TÀI LIỆU HỌC TẬP — DÀNH CHO SINH VIÊN

BUỔI 3: TRỰC QUAN HÓA & TIỀN XỬ LÝ DỮ LIỆU

Vẽ biểu đồ với Matplotlib & Chuẩn hóa/Mã hóa dữ liệu

Họ và tên: Nguyễn Quách Phú Tài	MSSV: 123000609	Nhóm: ……	Ngày: 29/09/2026

HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

- Tài liệu này đã có sẵn toàn bộ nội dung/code cho từng hoạt động — bạn không cần tự viết code từ đầu. Quy trình cho mỗi hoạt động là: 📋 Copy code có sẵn → ▶️ Paste vào Colab và chạy → 💭 Suy nghĩ/quan sát → 🔧 Cải tiến (tự thử thay đổi) → 🌾 Thu hoạch (ghi lại điều học được)
- Ở bước 🔧 Cải tiến, hãy dán phần code bạn đã tự chỉnh sửa vào khung viền vàng đứt nét; ở bước 🌾 Thu hoạch, hãy chụp màn hình (screenshot) kết quả (kể cả hình biểu đồ) và dán vào khung viền xám đứt nét (🖼️)
- Ghi câu trả lời/suy nghĩ của bạn vào các dòng kẻ chấm — không cần viết dài, chỉ cần thật (đây không phải bài kiểm tra đúng/sai)
- Cuối Tiết 4, hoàn thành phần "BÁO CÁO TRẢI NGHIỆM" ở cuối tài liệu này
- Trước khi rời lớp: vào File → Download → PDF Document để xuất toàn bộ tài liệu (đã có code cải tiến + ảnh + ghi chú của bạn) thành file PDF và nộp theo hướng dẫn của giảng viên
- ⚠️ Lưu ý: Ngay sau Buổi 3 sẽ có bài kiểm tra module "Nền tảng dữ liệu" (bao gồm nội dung Buổi 2–3) — hãy ôn tập kỹ các đoạn code đã cải tiến của bạn

MỤC TIÊU BUỔI HỌC

Sau buổi học này, bạn sẽ có khả năng:

- Chọn đúng loại biểu đồ (line, bar, scatter, histogram) phù hợp với từng loại câu hỏi/dữ liệu
- Vẽ được các biểu đồ cơ bản bằng matplotlib
- Giải thích được vì sao cần chuẩn hóa dữ liệu trước khi đưa vào mô hình AI
- Thực hiện được chuẩn hóa dữ liệu bằng scikit-learn (MinMaxScaler, StandardScaler)
- Mã hóa được biến phân loại bằng One-Hot Encoding (pandas.get_dummies)
- Thực hiện được quy trình đọc → làm sạch → chuẩn hóa → trực quan hóa trên một bộ dữ liệu thật

TIẾT 1 — TRẢI NGHIỆM: MỘT BỘ DỮ LIỆU, BỐN GÓC NHÌN

Mở notebook "Demo_Buoi3.ipynb" (đã có sẵn toàn bộ code bên dưới). Với mỗi demo, bạn chỉ cần: copy code → paste vào một cell Colab → nhấn Shift+Enter để chạy. Sau đó suy nghĩ, thử cải tiến, và ghi lại thu hoạch.

Demo 1: Bar chart — so sánh điểm trung bình theo môn

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import pandas as pd
import matplotlib.pyplot as plt

diem_lop = {
    "ten": ["An", "Bình", "Chi", "Dũng", "Em", "Phúc", "Giang", "Hoa"],
    "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0, 6.5, 9.5],
    "van":  [7.0, 8.0, 8.5, 6.5, 7.5, 9.0, 6.0, 8.0],
```

```
}
df = pd.DataFrame(diem_lop)

diem_tb_mon = df[["toan", "van"]].mean()
diem_tb_mon.plot(kind="bar", color=["#4C72B0", "#DD8452"])
plt.title("Điểm trung bình theo môn")
plt.ylabel("Điểm")
plt.show()
```

💭 **Suy nghĩ: Biểu đồ bar này giúp bạn trả lời câu hỏi gì?**

Bar chart giúp so sánh giá trị giữa các nhóm rời rạc. Ở đây nó trả lời: môn nào lớp học tốt hơn (điểm trung bình Toán so với Văn). Nhìn chiều cao cột là biết ngay môn nào cao hơn, không phải đọc từng dòng số trong bảng.

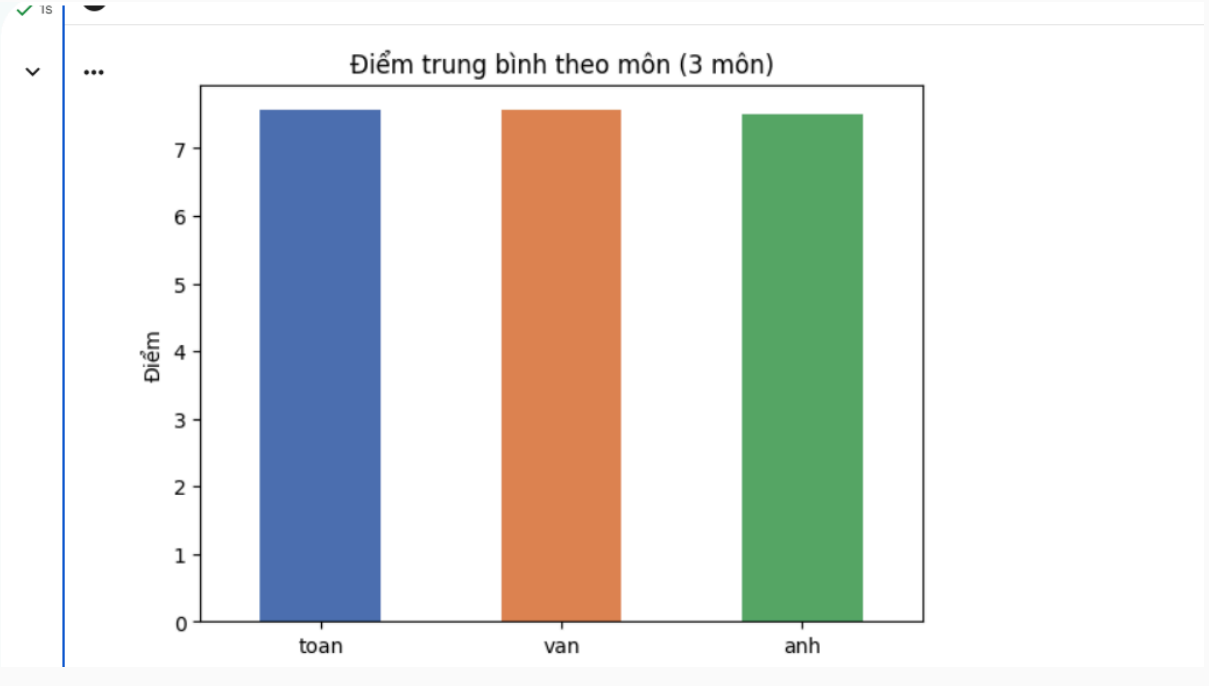
🔧 Cải tiến: Hãy thêm cột điểm môn Anh vào diem_lop (tự nghĩ số liệu) và vẽ lại bar chart so sánh cả 3 môn:

```
import pandas as pd
import matplotlib.pyplot as plt

diem_lop = {
   "ten": ["An", "Bình", "Chi", "Dũng", "Em", "Phúc", "Giang", "Hoa"],
   "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0, 6.5, 9.5],
   "van":  [7.0, 8.0, 8.5, 6.5, 7.5, 9.0, 6.0, 8.0],
   "anh":  [7.5, 6.5, 8.0, 7.0, 8.5, 7.5, 6.0, 9.0],
}
df = pd.DataFrame(diem_lop)

diem_tb_mon = df[["toan", "van", "anh"]].mean()
diem_tb_mon.plot(kind="bar", color=["#4C72B0", "#DD8452", "#55A868"])
plt.title("Điểm trung bình theo môn (3 môn)")
plt.ylabel("Điểm")
plt.xticks(rotation=0)
plt.show()
```

🌾 Thu hoạch: chụp màn hình biểu đồ sau khi cải tiến:



Demo 2: Scatter plot — mối quan hệ giữa 2 biến

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
plt.scatter(df["toan"], df["van"])
plt.xlabel("Điểm Toán")
plt.ylabel("Điểm Văn")
plt.title("Mối quan hệ giữa điểm Toán và điểm Văn")
plt.show()
```

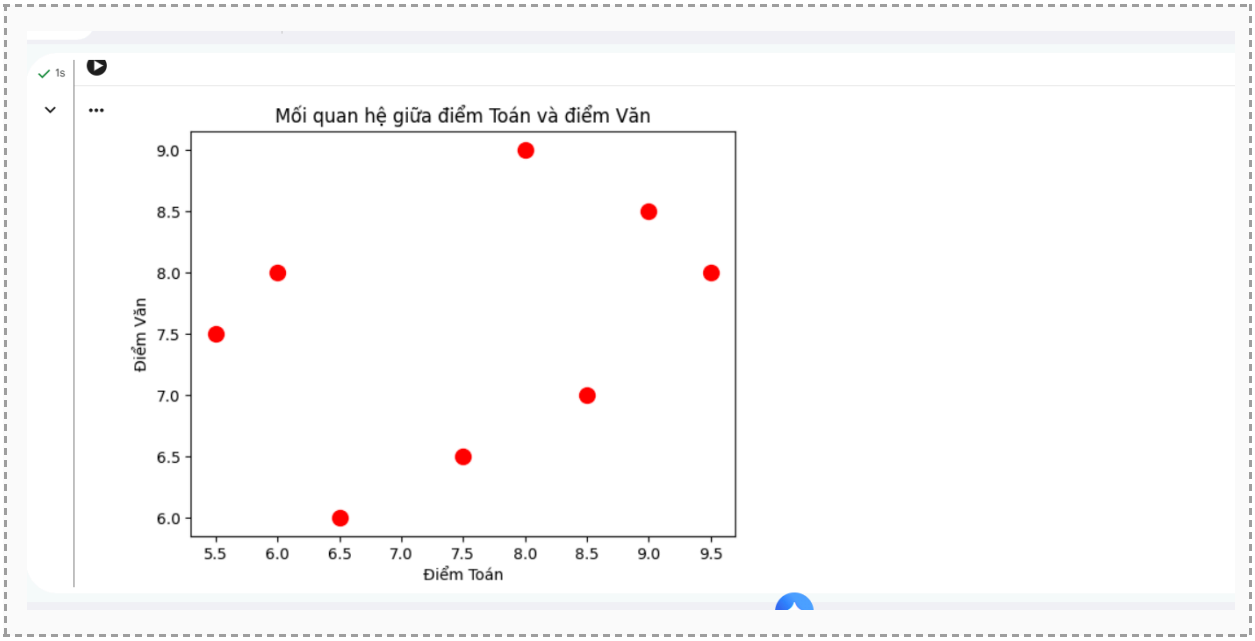
💭 **Suy nghĩ: Nhìn vào scatter plot, bạn có thấy điểm Toán cao thì điểm Văn cũng có xu hướng cao không?**

Không rõ ràng. Các điểm rải khá rác: có bạn Toán cao nhưng Văn chỉ ở mức khá, có bạn Toán thấp hơn nhưng Văn cao (ví dụ Bình). Không thấy đường chéo tăng rõ. Với 8 điểm thì chưa đủ kết luận tương quan dương mạnh.

🔧 Cải tiến: Hãy đổi màu và kích thước điểm trên scatter plot (gợi ý: thêm tham số color="red", s=100 vào plt.scatter), rồi chạy lại:

```
plt.scatter(df["toan"], df["van"], color="red", s=100)
plt.xlabel("Điểm Toán")
plt.ylabel("Điểm Văn")
plt.title("Mối quan hệ giữa điểm Toán và điểm Văn")
plt.show()
```

🌾 Thu hoạch: chụp màn hình scatter plot sau khi cải tiến:



Demo 3: Histogram — phân bố một biến

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
plt.hist(df["toan"], bins=5, color="#55A868", edgecolor="black")
plt.xlabel("Điểm Toán")
plt.ylabel("Số lượng sinh viên")
plt.title("Phân bố điểm Toán của lớp")
plt.show()
```

💭 **Suy nghĩ: bins=5 có nghĩa là gì? Nếu đổi thành bins=10 thì biểu đồ sẽ khác ra sao?**

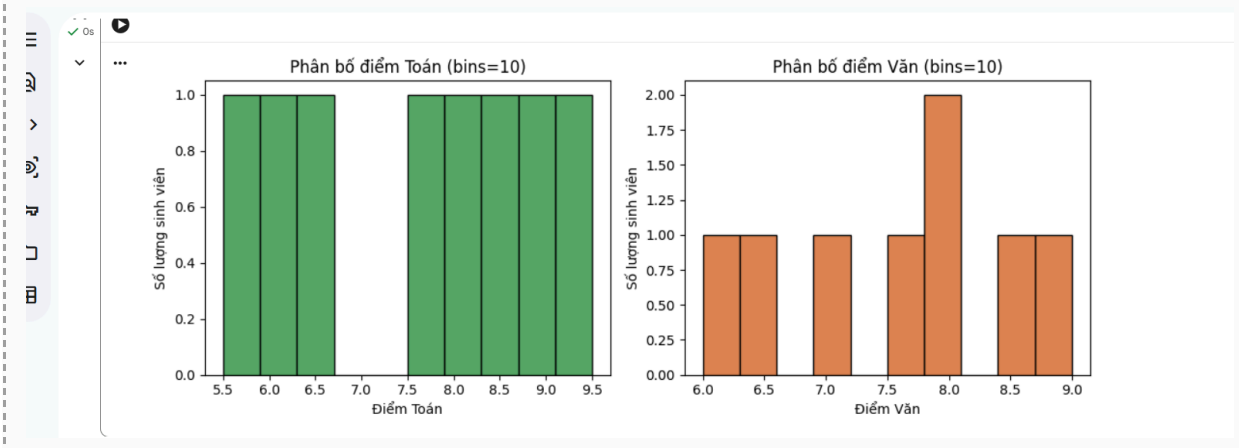
bins là số khoảng chia trên trục điểm. bins=5 gom dữ liệu thành 5 cột → nhìn khái quát, dễ thấy khối chính. bins=10 chia nhỏ hơn → thấy chi tiết hơn nhưng với chỉ 8 sinh viên nhiều cột sẽ trống, biểu đồ trông rời rạc hơn.

🔧 Cải tiến: Hãy đổi bins=5 thành bins=10, và vẽ thêm histogram cho cột điểm Văn:

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].hist(df["toan"], bins=10, color="#55A868", edgecolor="black")
axes[0].set_xlabel("Điểm Toán")
axes[0].set_ylabel("Số lượng sinh viên")
axes[0].set_title("Phân bố điểm Toán (bins=10)")

axes[1].hist(df["van"], bins=10, color="#DD8452", edgecolor="black")
axes[1].set_xlabel("Điểm Văn")
axes[1].set_ylabel("Số lượng sinh viên")
axes[1].set_title("Phân bố điểm Văn (bins=10)")
plt.tight_layout()
plt.show()
```

🌾 Thu hoạch: chụp màn hình histogram sau khi cải tiến:



Demo 4: Line chart — xu hướng theo thời gian

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
tuan = [1, 2, 3, 4, 5, 6]
diem_tb_tuan = [6.8, 7.0, 7.2, 7.5, 7.3, 7.8]

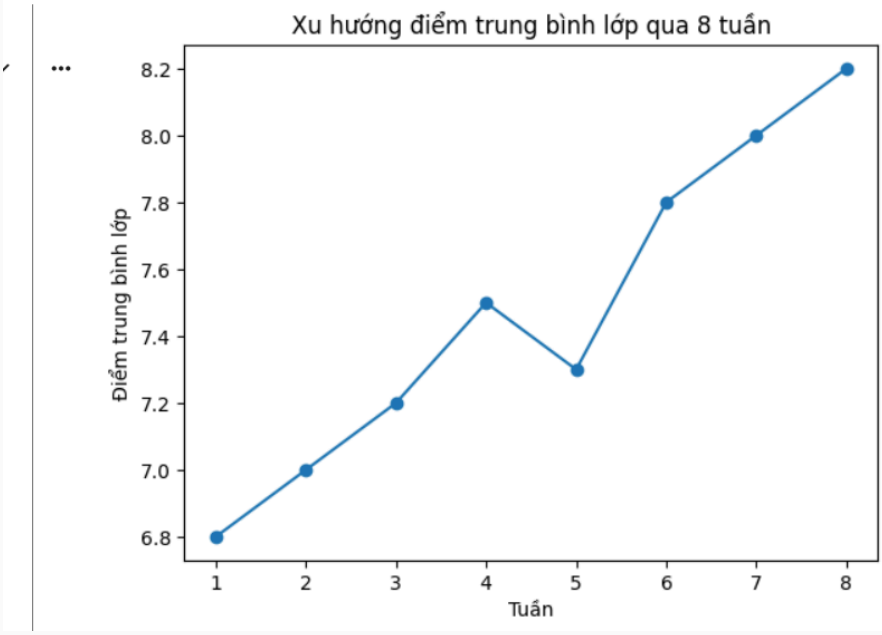
plt.plot(tuan, diem_tb_tuan, marker="o")
plt.xlabel("Tuần")
plt.ylabel("Điểm trung bình lớp")
plt.title("Xu hướng điểm trung bình lớp qua các tuần")
plt.show()
```

🔧 Cải tiến: Hãy thêm dữ liệu tuần 7, 8 (tự nghĩ số liệu) vào danh sách và vẽ lại đường xu hướng:

```
tuan = [1, 2, 3, 4, 5, 6, 7, 8]
diem_tb_tuan = [6.8, 7.0, 7.2, 7.5, 7.3, 7.8, 8.0, 8.2]

plt.plot(tuan, diem_tb_tuan, marker="o")
plt.xlabel("Tuần")
plt.ylabel("Điểm trung bình lớp")
plt.title("Xu hướng điểm trung bình lớp qua 8 tuần")
plt.show()
```

🌾 Thu hoạch: chụp màn hình line chart sau khi cải tiến:



Câu hỏi thảo luận: Trong 4 biểu đồ vừa vẽ, biểu đồ nào phù hợp để trả lời câu hỏi 'lớp có tiến bộ theo thời gian không?'; biểu đồ nào phù hợp để trả lời 'môn nào lớp học tốt hơn?'

Tiến bộ theo thời gian → line chart (trục X là tuần, nhìn dốc là biết tăng/giảm). Môn nào học tốt hơn → bar chart (so sánh điểm TB giữa các môn). Scatter dùng cho quan hệ 2 biến số; histogram dùng cho phân bố 1 biến.

TIẾT 2 — KHÁI QUÁT HÓA: GHI CHÚ LÝ THUYẾT

Vừa nghe giảng vừa điền vào các mục dưới đây bằng lời của chính bạn (không chép nguyên văn slide):

Khi nào nên dùng line chart? Khi nào nên dùng bar chart? Khi nào nên dùng scatter plot? Khi nào nên dùng histogram?

Line chart: dữ liệu theo thời gian hoặc thứ tự, muốn thấy xu hướng (điểm TB theo tuần, doanh thu theo tháng). Bar chart: so sánh giữa các nhóm rời rạc (môn học, danh mục sản phẩm, nam/nữ). Scatter plot: hai biến số, hỏi có mối quan hệ/tương quan không (giá vs số lượng, Toán vs Văn). Histogram: phân bố của một biến số — dữ liệu tập trung ở đâu, lệch trái/phải, có khoảng trống không.

Chuẩn hóa dữ liệu (Min-Max Scaling / Standard Scaling) là gì? Vì sao cần làm điều này trước khi đưa dữ liệu vào mô hình AI?

Min-Max Scaling kéo mỗi cột về khoảng [0, 1] theo công thức (x − min) / (max − min). Standard Scaling đưa về trung bình 0, độ lệch chuẩn 1: (x − mean) / std. Cần làm vì các cột thường khác đơn vị và khác biên độ (điểm 0–10 vs giờ tự học 2–15 vs giá 3.000–85.000). Mô hình dùng khoảng cách hoặc gradient (KNN, SVM, mạng nơ-ron, hồi quy có regularize) sẽ bị cột có số lớn 'át' cột có số nhỏ. Chuẩn hóa để mỗi đặc trưng đóng góp công bằng.

One-Hot Encoding là gì? Vì sao mô hình AI cần bước này với dữ liệu dạng chữ?

One-Hot biến 1 cột chữ thành nhiều cột 0/1, mỗi cột ứng với một giá trị (lớp A1 → lop_A1=1, lop_A2=0). Mô hình toán học không hiểu chuỗi 'A1'/'A2'. Nếu gán số tùy tiện (A1=0, A2=1, A3=2) máy có thể hiểu nhầm A3 'lớn hơn' A1 dù đó chỉ là nhãn. One-Hot tránh giả định thứ tự giả tạo đó.

TIẾT 3 — ÁP DỤNG CÓ HƯỚNG DẪN: CHUẨN HÓA & MÃ HÓA DỮ LIỆU

📋 Nội dung có sẵn — Chuẩn hóa dữ liệu với MinMaxScaler. Copy và chạy:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

diem_lop3 = {
    "ten": ["An", "Bình", "Chi", "Dũng", "Em", "Phúc"],
    "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0],
    "gio_tu_hoc": [10, 3, 15, 8, 2, 12],   # số giờ tự học/tuần
}
df3 = pd.DataFrame(diem_lop3)

scaler = MinMaxScaler()
df3[["toan_scaled",   "gio_scaled"]]   =   scaler.fit_transform(df3[["toan",
"gio_tu_hoc"]])
print(df3)
```

💭 **Suy nghĩ: Giá trị toan_scaled và gio_scaled nằm trong khoảng nào? Vì sao 2 cột có đơn vị khác nhau (điểm vs giờ) lại về cùng một khoảng?**

Với MinMaxScaler cả hai cột nằm trong [0, 1]. Em có toan=5.5 và giờ=2 ⟶ scaled = 0 (min). Chi có toan=9.0 và giờ=15 ⟶ scaled = 1 (max). Hai cột khác đơn vị nhưng sau khi chia cho (max−min) của chính cột đó, biên độ bị kéo về cùng thước đo 0–1 nên mô hình không bị cột 'giờ' hoặc cột 'giá' đè.

🔧 Cải tiến: Hãy đổi MinMaxScaler thành StandardScaler (cùng import từ sklearn.preprocessing), chạy lại và so sánh giá trị chuẩn hóa khác nhau như thế nào:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

diem_lop3 = {
   "ten": ["An", "Bình", "Chi", "Dũng", "Em", "Phúc"],
   "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0],
   "gio_tu_hoc": [10, 3, 15, 8, 2, 12],
}
df3 = pd.DataFrame(diem_lop3)

scaler = StandardScaler()
df3[["toan_scaled", "gio_scaled"]] = scaler.fit_transform(df3[["toan", "gio_tu_hoc"]])
print(df3)
print("mean_:", scaler.mean_)
print("scale_:", scaler.scale_)
```

🌾 Thu hoạch: chụp màn hình kết quả sau khi đổi sang StandardScaler:

```
from sklearn.preprocessing import StandardScaler

diem_lop3 = {
    "ten": ["An", "Bình", "Chi", "Dũng", "Em", "Phúc"],
    "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0],
    "gio_tu_hoc": [10, 3, 15, 8, 2, 12],
}
df3 = pd.DataFrame(diem_lop3)

scaler = StandardScaler()
df3[["toan_scaled", "gio_scaled"]] = scaler.fit_transform(df3[["toan", "gio_tu_hoc"]])
print(df3)
print("mean_:", scaler.mean_)
print("scale_:", scaler.scale_)
```

```
     ten  toan  gio_tu_hoc  toan_scaled  gio_scaled
0     An   8.5          10     0.851658    0.358979
1   Bình   6.0           3    -1.113707   -1.148733
2    Chi   9.0          15     1.244731    1.435916
3   Dũng   7.5           8     0.065512   -0.071796
4     Em   5.5           2    -1.506780   -1.364121
5   Phúc   8.0          12     0.458585    0.789754
mean_: [7.41666667 8.33333333]
scale_: [1.27202813 4.64279609]
```

📋 Nội dung có sẵn — Mã hóa One-Hot với pandas.get_dummies. Copy và chạy:

```
import pandas as pd

diem_lop4 = {
    "ten": ["An", "Bình", "Chi", "Dũng"],
    "lop": ["A1", "A2", "A1", "A2"],
    "toan": [8.5, 6.0, 9.0, 7.5],
}
df4 = pd.DataFrame(diem_lop4)

df4_encoded = pd.get_dummies(df4, columns=["lop"])
print(df4_encoded)
```

💭 **Suy nghĩ: Cột "lop" đã biến thành những cột mới nào? Giá trị trong các cột mới đó là gì?**

Cột lop biến thành lop_A1 và lop_A2. Mỗi dòng chỉ một cột bằng True/1, cột còn lại False/0. An học A1 nên lop_A1=True, lop_A2=False. Không còn cột chữ 'lop' nếu dùng get_dummies đúng cách.

🔧 Cải tiến: Hãy thêm một cột phân loại mới (VD: "gioi_tinh" với giá trị "Nam"/"Nữ") vào diem_lop4 và mã hóa One-Hot đồng thời cả 2 cột "lop" và "gioi_tinh":

```
import pandas as pd

diem_lop4 = {
   "ten": ["An", "Bình", "Chi", "Dũng"],
   "lop": ["A1", "A2", "A1", "A2"],
   "gioi_tinh": ["Nam", "Nam", "Nữ", "Nữ"],
   "toan": [8.5, 6.0, 9.0, 7.5],
}
df4 = pd.DataFrame(diem_lop4)

df4_encoded = pd.get_dummies(df4, columns=["lop", "gioi_tinh"])
print(df4_encoded)
```

🌾 Thu hoạch: chụp màn hình kết quả sau khi cải tiến:

```
        "gioi_tinh": ["Nam", "Nam", "Nữ", "Nữ"],
        "toan": [8.5, 6.0, 9.0, 7.5],
    }
    df4 = pd.DataFrame(diem_lop4)

    df4_encoded = pd.get_dummies(df4, columns=["lop", "gioi_tinh"])
    print(df4_encoded)
```

```
    ten  toan  lop_A1  lop_A2  gioi_tinh_Nam  gioi_tinh_Nữ
0    An   8.5    True   False           True         False
1  Bình   6.0   False    True           True         False
2   Chi   9.0    True   False          False          True
3  Dũng   7.5   False    True          False          True
```

TIẾT 4 — ÁP DỤNG MỞ RỘNG: QUY TRÌNH ĐẦY ĐỦ TRÊN DỮ LIỆU BÁN HÀNG

Làm việc theo nhóm 2–3 người.

📋 Nội dung có sẵn — copy và chạy đoạn code thực hiện quy trình đầy đủ: làm sạch → chuẩn hóa → trực quan hóa:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

ban_hang = {
     "san_pham": ["Gạo", "Đường", "Sữa", "Nước mắm", "Mì gói", "Trứng", "Dầu
ăn", "Rau"],
      "danh_muc": ["Lương thực", "Thực phẩm", "Thực phẩm", "Gia vị", "Lương
thực", "Thực phẩm", "Gia vị", "Rau củ"],
    "gia_ban": [18000, 22000, np.nan, 35000, 4000, 3000, 45000, 12000],
    "so_luong_ban": [120, 80, 60, 90, 200, 300, np.nan, 150],
}
df5 = pd.DataFrame(ban_hang)

# Bước 1: Làm sạch — xử lý giá trị thiếu bằng trung bình cột
df5["gia_ban"] = df5["gia_ban"].fillna(df5["gia_ban"].mean())
df5["so_luong_ban"] = df5["so_luong_ban"].fillna(df5["so_luong_ban"].mean())

# Bước 2: Chuẩn hóa
scaler = MinMaxScaler()
df5[["gia_ban_scaled",                    "so_luong_scaled"]]                =
scaler.fit_transform(df5[["gia_ban", "so_luong_ban"]])
print(df5)

# Bước 3: Trực quan hóa
df5.groupby("danh_muc")["gia_ban"].mean().plot(kind="bar", color="#4C72B0")
plt.title("Giá bán trung bình theo danh mục")
plt.ylabel("Giá bán (VNĐ)")
plt.show()

plt.scatter(df5["gia_ban"], df5["so_luong_ban"])
plt.xlabel("Giá bán")
plt.ylabel("Số lượng bán")
plt.title("Mối quan hệ giữa giá bán và số lượng bán")
plt.show()
```

✓ Chạy toàn bộ code trên và hiểu rõ 3 bước: làm sạch, chuẩn hóa, trực quan hóa

🔧 Cải tiến: Cả nhóm cùng thêm ít nhất 2 sản phẩm mới vào dữ liệu (có thể để trống một vài giá trị để luyện làm sạch), chạy lại toàn bộ quy trình 3 bước, và vẽ thêm 1 biểu đồ loại khác (histogram hoặc line) chưa được dùng ở trên:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

ban_hang = {
   "san_pham": ["Gạo", "Đường", "Sữa", "Nước mắm", "Mì gói",
            "Trứng", "Dầu ăn", "Rau", "Cà phê", "Bánh mì"],
   "danh_muc": ["Lương thực", "Thực phẩm", "Thực phẩm", "Gia vị",
            "Lương thực", "Thực phẩm", "Gia vị", "Rau củ",
            "Thực phẩm", "Lương thực"],
   "gia_ban": [18000, 22000, np.nan, 35000, 4000,
           3000, 45000, 12000, 85000, np.nan],
   "so_luong_ban": [120, 80, 60, 90, 200,
               300, np.nan, 150, 40, 180],
}
df5 = pd.DataFrame(ban_hang)

# Bước 1: Làm sạch
df5["gia_ban"] = df5["gia_ban"].fillna(df5["gia_ban"].mean())
df5["so_luong_ban"] = df5["so_luong_ban"].fillna(df5["so_luong_ban"].mean())

# Bước 2: Chuẩn hóa
scaler = MinMaxScaler()
df5[["gia_ban_scaled", "so_luong_scaled"]] = scaler.fit_transform(
   df5[["gia_ban", "so_luong_ban"]]
)
print(df5)

# Bước 3: Trực quan hóa
df5.groupby("danh_muc")["gia_ban"].mean().plot(kind="bar", color="#4C72B0")
plt.title("Giá bán trung bình theo danh mục")
plt.ylabel("Giá bán (VNĐ)")
plt.show()

plt.scatter(df5["gia_ban"], df5["so_luong_ban"])
plt.xlabel("Giá bán")
plt.ylabel("Số lượng bán")
plt.title("Mối quan hệ giữa giá bán và số lượng bán")
plt.show()

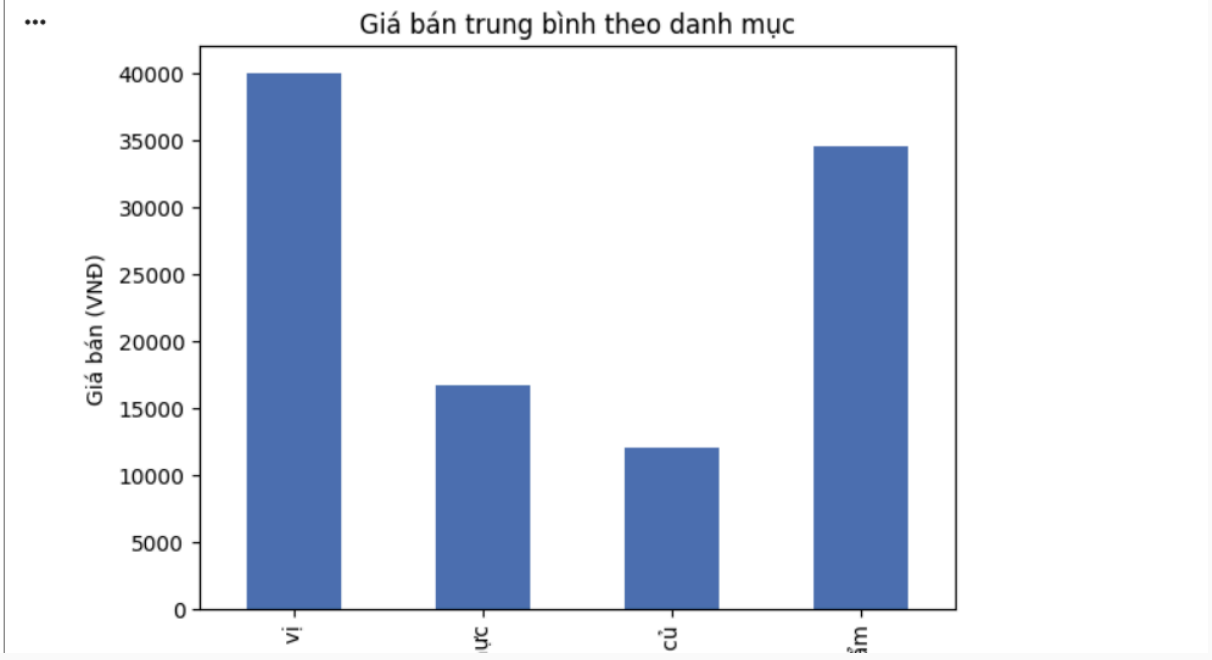
# Biểu đồ mới: histogram phân bố giá
plt.hist(df5["gia_ban"], bins=6, color="#55A868", edgecolor="black")
plt.xlabel("Giá bán (VNĐ)")
plt.ylabel("Số sản phẩm")
plt.title("Phân bố giá bán sau khi làm sạch")
plt.show()
```

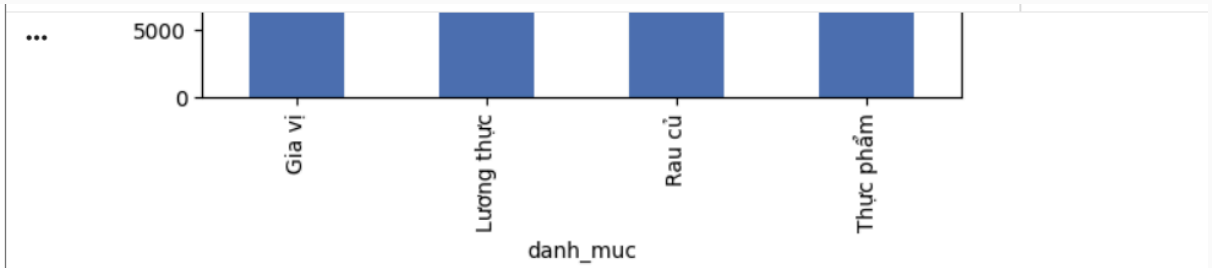
🌾 Thu hoạch: chụp màn hình kết quả bảng dữ liệu đã xử lý và biểu đồ mới nhóm vừa vẽ:

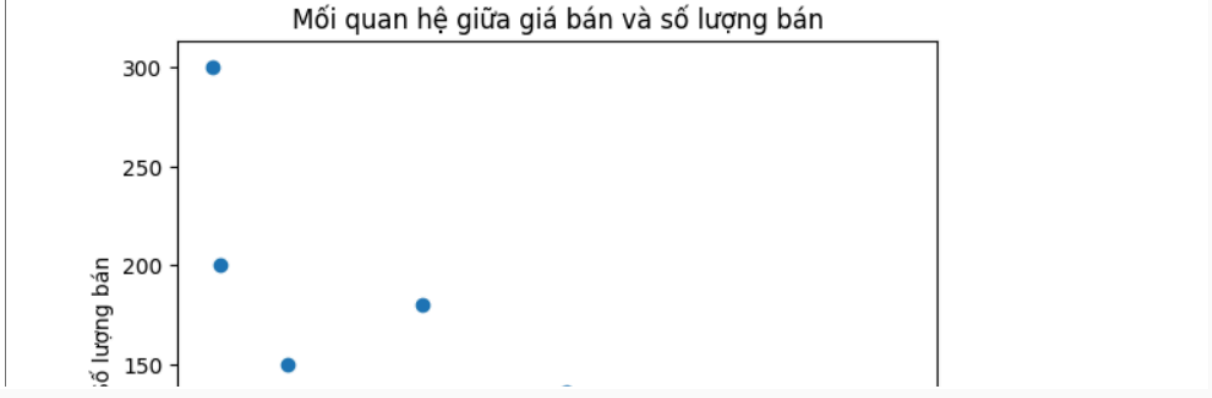
```
    san_pham    danh_muc  gia_ban  so_luong_ban  gia_ban_scaled  \
0       Gạo  Lương thực  18000.0    120.000000        0.182927
1     Đường   Thực phẩm  22000.0     80.000000        0.231707
2       Sữa   Thực phẩm  28000.0     60.000000        0.304878
3  Nước mắm     Gia vị  35000.0     90.000000        0.390244
4    Mì gói  Lương thực   4000.0    200.000000        0.012195
5     Trứng   Thực phẩm   3000.0    300.000000        0.000000
6    Dầu ăn     Gia vị  45000.0    135.555556        0.512195
7       Rau     Rau củ  12000.0    150.000000        0.109756
8    Cà phê   Thực phẩm  85000.0     40.000000        1.000000
9   Bánh mì  Lương thực  28000.0    180.000000        0.304878

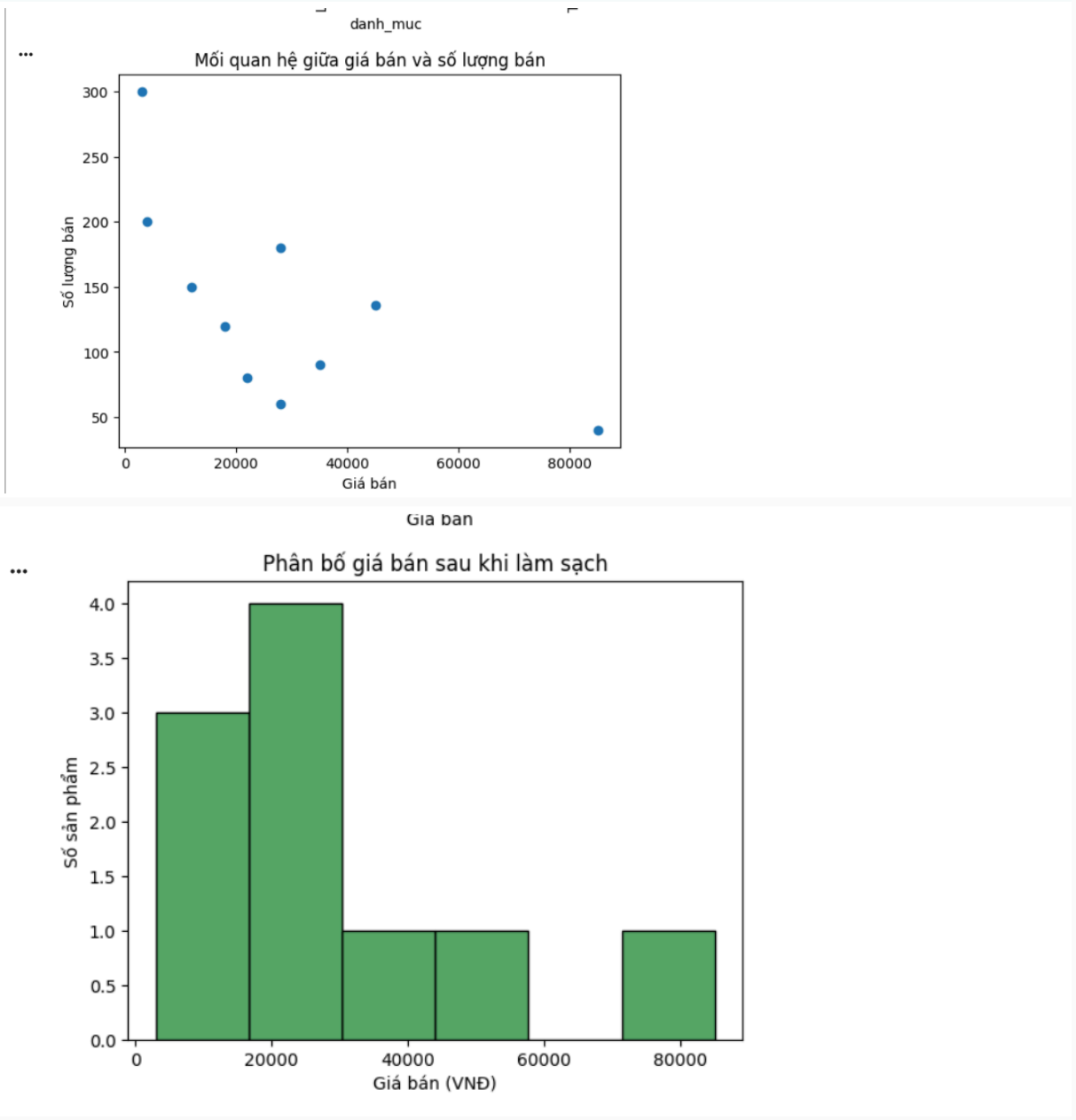
   so_luong_scaled
0         0.307692
1         0.153846
2         0.076923
3         0.192308
4         0.615385
5         1.000000
6         0.367521
7         0.423077
8         0.000000
9         0.538462
```

Giá bán trung bình theo danh mục









Phát hiện thú vị nhất mà nhóm bạn tìm được từ biểu đồ mới:

Scatter cho thấy xu hướng giá cao thì số lượng bán thấp (Cà phê 85.000 chỉ bán 40; Trứng 3.000 bán 300). Histogram giá lệch phải: phần lớn mặt hàng giá thấp–trung bình, một sản phẩm giá rất cao kéo max của MinMaxScaler. Hệ quả: sau scale, Cà phê = 1.0 còn Mì gói/Trứng gần 0 — outlier giá làm các sản phẩm còn lại bị 'dồn' về phía 0. Nếu dùng StandardScaler, Cà phê sẽ là điểm z-score lớn, dễ thấy hơn là điểm bất thường.

3 BÀI THỰC HÀNH THÊM

Làm thêm 3 bài thực hành sau để củng cố kỹ năng xử lý & trực quan hóa dữ liệu (có thể làm ở Tiết 3 nếu còn thời gian, hoặc làm ở nhà trước bài kiểm tra module).

1 — Phát hiện Outlier bằng Boxplot & IQR

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt

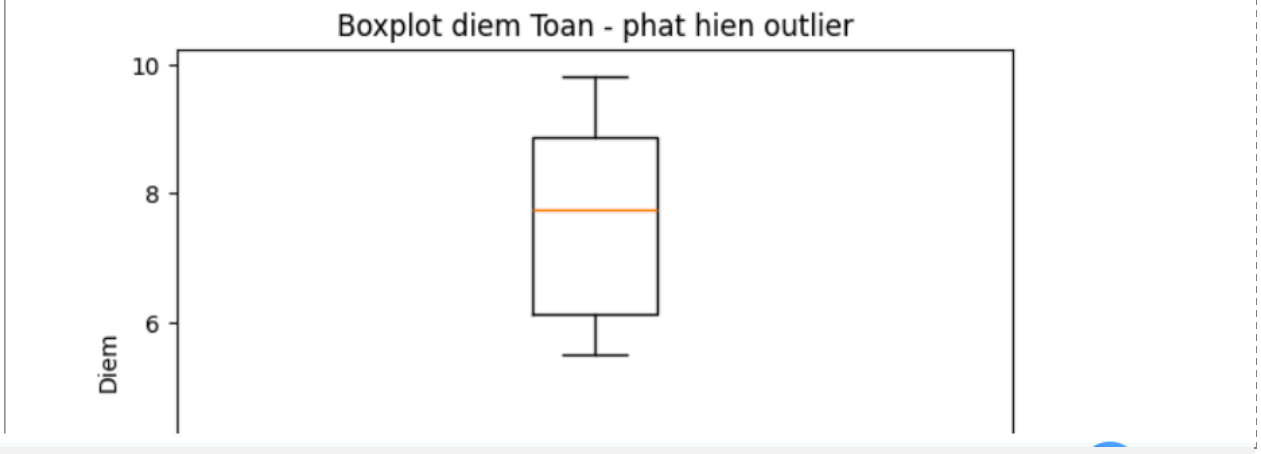
diem_lop5 = {
   "ten": ["An","Binh","Chi","Dung","Em","Phuc","Giang","Hoa","Kien","Lan"],
   "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0, 6.5, 9.5, 1.0, 9.8],  # Kien=1.0 la diem bat thuong
}
df5 = pd.DataFrame(diem_lop5)
print(df5)

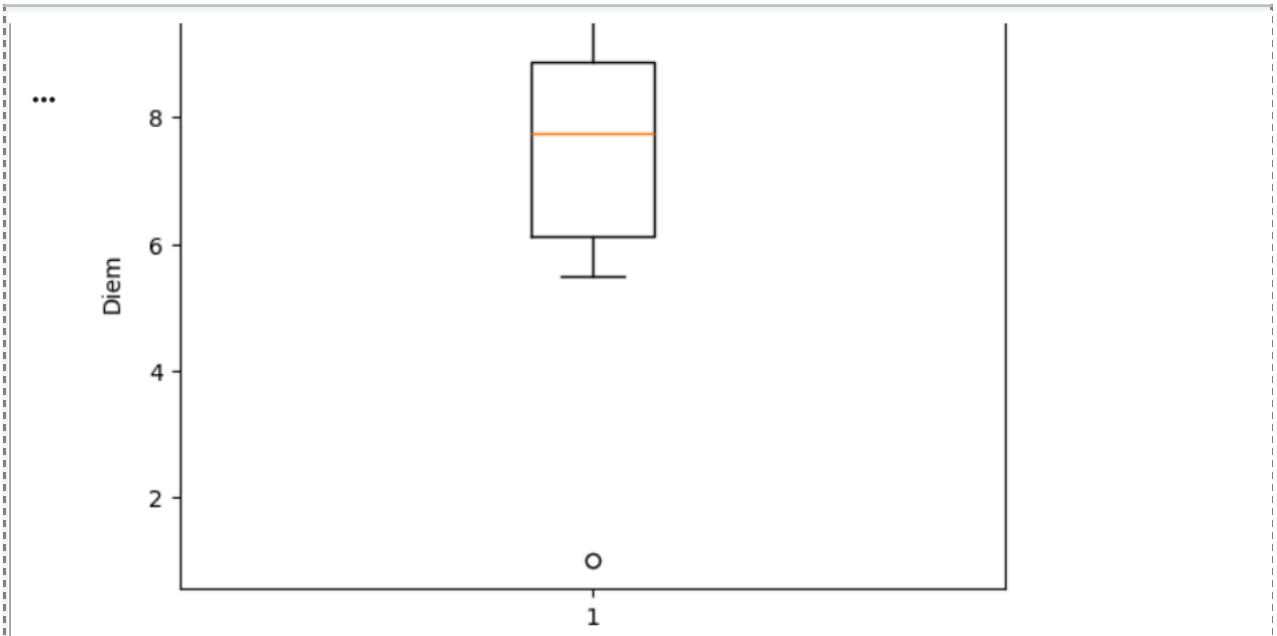
plt.boxplot(df5["toan"])
plt.title("Boxplot diem Toan - phat hien outlier")
plt.ylabel("Diem")
plt.show()

Q1 = df5["toan"].quantile(0.25)
Q3 = df5["toan"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print("Q1:", Q1, "| Q3:", Q3, "| IQR:", IQR)
print("Nguong duoi:", lower, "| Nguong tren:", upper)

outliers = df5[(df5["toan"] < lower) | (df5["toan"] > upper)]
print("Cac ban ghi bi coi la outlier:")
print(outliers)
```

```
...     ten  toan
    0    An   8.5
    1  Binh   6.0
    2   Chi   9.0
    3  Dung   7.5
    4    Em   5.5
    5  Phuc   8.0
    6 Giang   6.5
    7   Hoa   9.5
    8  Kien   1.0
    9   Lan   9.8
```






```
Q1: 6.125 | Q3: 8.875 | IQR: 2.75
Nguong duoi: 2.0 | Nguong tren: 13.0
Cac ban ghi bi coi la outlier:
    ten  toan
8  Kien   1.0
```

💭 **Suy nghĩ: Vì sao boxplot lại thể hiện trực quan các điểm "bất thường" tốt hơn nhìn bảng số liệu thô?**

Boxplot tóm tắt Q1–trung vị–Q3 và vẽ râu 1.5×IQR; điểm nằm ngoài râu bị chấm riêng nên 'lòi' ra ngay. Nhìn bảng 10 số, điểm 1.0 của Kiên dễ bị trượt mắt. Boxplot còn cho thấy độ phân tán và độ lệch của cả lớp, không chỉ một dòng.

🔧 Cải tiến: loại bỏ các dòng outlier khỏi df5 (dùng điều kiện ngược lại của lower/upper), in ra df5 sau khi lọc sạch, và vẽ lại boxplot để kiểm tra: một dòng.

```
import pandas as pd
import matplotlib.pyplot as plt

diem_lop5 = {
   "ten": ["An","Binh","Chi","Dung","Em","Phuc","Giang","Hoa","Kien","Lan"],
   "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0, 6.5, 9.5, 1.0, 9.8],
}
df5 = pd.DataFrame(diem_lop5)

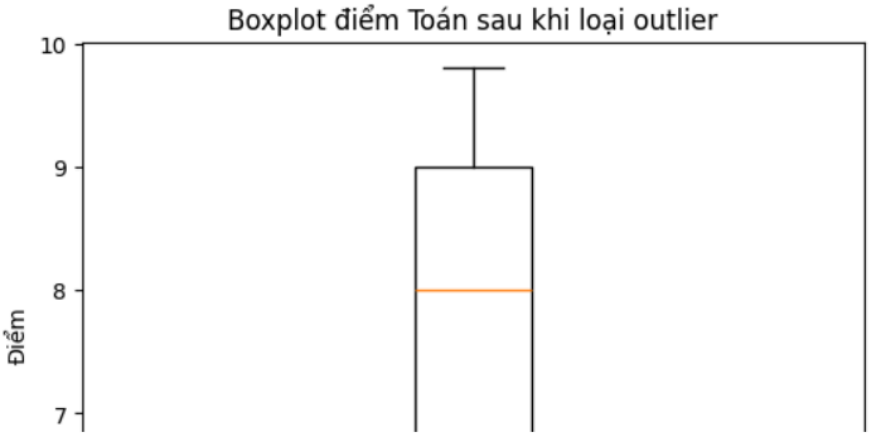
Q1 = df5["toan"].quantile(0.25)
Q3 = df5["toan"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

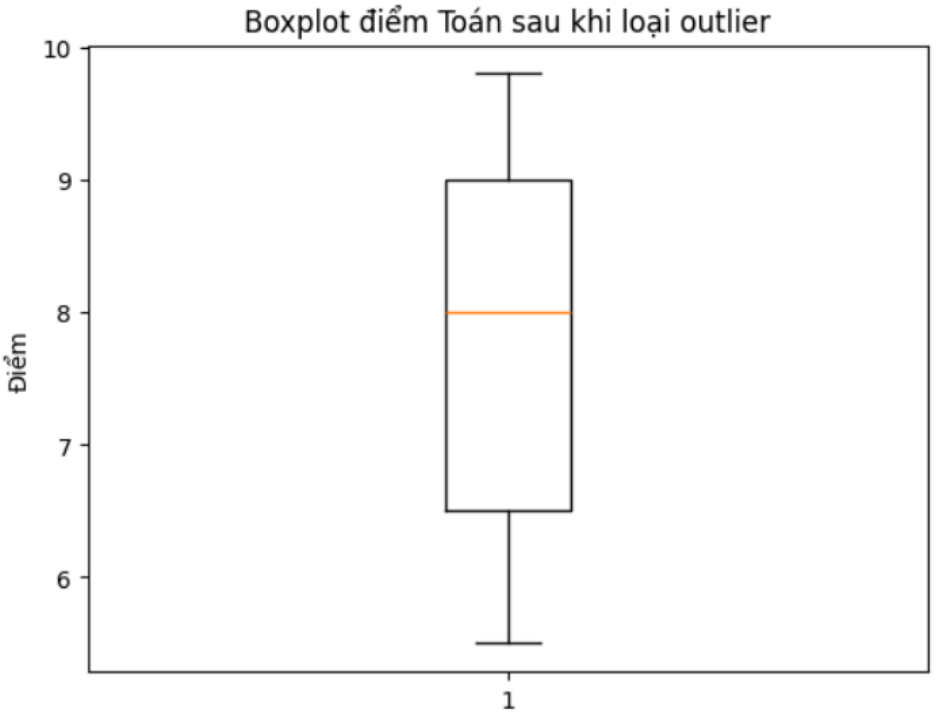
df5_clean = df5[(df5["toan"] >= lower) & (df5["toan"] <= upper)]
print("Sau khi lọc outlier:")
print(df5_clean)

plt.boxplot(df5_clean["toan"])
plt.title("Boxplot điểm Toán sau khi loại outlier")
plt.ylabel("Điểm")
```

```
plt.show()
```

```
Sau khi lọc outlier:
    ten  toan
0    An   8.5
1  Binh   6.0
2   Chi   9.0
3  Dung   7.5
4    Em   5.5
5  Phuc   8.0
6 Giang   6.5
7   Hoa   9.5
9   Lan   9.8
```





2 — So sánh Label Encoding và One-Hot Encoding

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

xep_loai = {
   "ten": ["An","Binh","Chi","Dung","Em"],
   "xep_loai": ["Kha","Gioi","Trung binh","Gioi","Xuat sac"],
}
df6 = pd.DataFrame(xep_loai)
print("Du lieu goc:")
print(df6)

le = LabelEncoder()
df6["xep_loai_label"] = le.fit_transform(df6["xep_loai"])
print("Sau Label Encoding:")
print(df6)
print("Anh xa nhan:", dict(zip(le.classes_, le.transform(le.classes_))))

df6_onehot = pd.get_dummies(df6[["ten","xep_loai"]], columns=["xep_loai"])
print("Sau One-Hot Encoding:")
print(df6_onehot)
```

```
Du lieu goc:
    ten    xep_loai
0    An         Kha
1  Binh        Gioi
2   Chi  Trung binh
3  Dung        Gioi
4    Em    Xuat sac
Sau Label Encoding:
    ten    xep_loai  xep_loai_label
0    An         Kha               1
1  Binh        Gioi               0
2   Chi  Trung binh               2
3  Dung        Gioi               0
4    Em    Xuat sac               3
Anh xa nhan: {'Gioi': np.int64(0), 'Kha': np.int64(1), 'Trung binh': np.int64(2), 'Xuat sac': np.int64(3)}
Sau One-Hot Encoding:
    ten  xep_loai_Gioi  xep_loai_Kha  xep_loai_Trung binh  xep_loai_Xuat sac
0    An          False          True                False              False
1  Binh           True         False                False              False
2   Chi          False         False                 True              False
3  Dung           True         False                False              False
4    Em          False         False                False               True
```

💭 **Suy nghĩ: LabelEncoder gán số theo thứ tự bảng chữ cái (Giỏi=0, Khá=1, Trung bình=2, Xuất sắc=3). Thứ tự này có đúng với thứ hạng học lực thật không? Nếu đưa cột này vào một mô hình hồi quy, điều gì có thể xảy ra?**

Không đúng. Alphabet: Gioi < Kha < Trung binh < Xuat sac, trong khi thứ hạng thật phải là Trung bình < Khá < Giỏi < Xuất sắc. Nếu nhét cột số sai thứ tự vào hồi quy/linear model, máy hiểu 'Giỏi < Khá' và 'khoảng cách Giỏi–Xuất sắc = 3' — cả hai đều sai. One-Hot không giả định thứ tự nên an toàn hơn với nhãn không có hạng. Với nhãn CÓ hạng, nên dùng Categorical ordered hoặc map tay 0–1–2–3 đúng thứ tự.

🔧 Cải tiến: tạo danh sách thu_tu_dung = ["Trung binh","Kha","Gioi","Xuat sac"] rồi dùng pd.Categorical(df6["xep_loai"], categories=thu_tu_dung, ordered=True).codes để mã hóa lại ĐÚNG thứ tự học lực. So sánh với kết quả LabelEncoder ở trên:

✏️ *Dán code mã hóa theo đúng thứ tự học lực (pd.Categorical) của bạn vào đây*

```
import pandas as pd


xep_loai = {
   "ten": ["An","Binh","Chi","Dung","Em"],
   "xep_loai": ["Kha","Gioi","Trung binh","Gioi","Xuat sac"],
}
df6 = pd.DataFrame(xep_loai)

thu_tu_dung = ["Trung binh", "Kha", "Gioi", "Xuat sac"]
df6["xep_loai_ordered"] = pd.Categorical(
   df6["xep_loai"], categories=thu_tu_dung, ordered=True
).codes
print(df6)
print("Ánh xạ đúng thứ tự:", dict(enumerate(thu_tu_dung)))
```

```
    ten    xep_loai  xep_loai_ordered
0    An         Kha                 1
1  Binh        Gioi                 2
2   Chi  Trung binh                 0
3  Dung        Gioi                 2
4    Em    Xuat sac                 3
Ánh xạ đúng thứ tự: {0: 'Trung binh', 1: 'Kha', 2: 'Gioi', 3: 'Xuat sac'}
```

Cải tiến: đổi `MinMaxScaler` thành `StandardScaler`, chạy lại và so sánh giá trị:

3 — Dashboard: 4 biểu đồ trong 1 hình bằng plt.subplots

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import pandas as pd
import matplotlib.pyplot as plt

diem_lop = {
   "ten": ["An","Binh","Chi","Dung","Em","Phuc","Giang","Hoa"],
   "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0, 6.5, 9.5],
   "van":  [7.0, 8.0, 8.5, 6.5, 7.5, 9.0, 6.0, 8.0],
}
df = pd.DataFrame(diem_lop)

fig, axes = plt.subplots(2, 2, figsize=(10, 8))

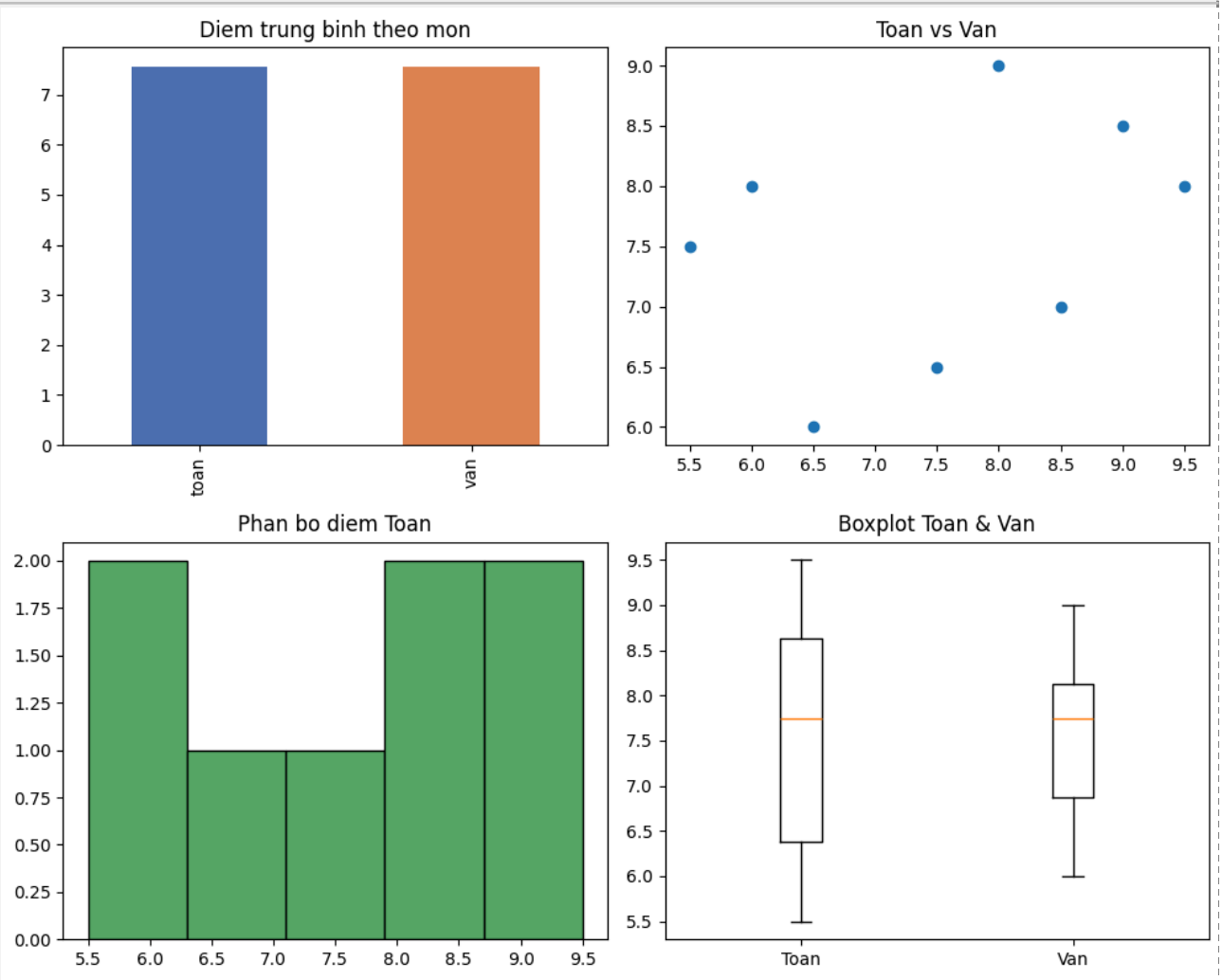
df[["toan","van"]].mean().plot(kind="bar", ax=axes[0,0], color=["#4C72B0","#DD8452"])
axes[0,0].set_title("Diem trung binh theo mon")

axes[0,1].scatter(df["toan"], df["van"])
axes[0,1].set_title("Toan vs Van")

axes[1,0].hist(df["toan"], bins=5, color="#55A868", edgecolor="black")
axes[1,0].set_title("Phan bo diem Toan")

axes[1,1].boxplot([df["toan"], df["van"]])
axes[1,1].set_xticks([1, 2])
axes[1,1].set_xticklabels(["Toan", "Van"])
axes[1,1].set_title("Boxplot Toan & Van")

plt.tight_layout()
plt.show()
```



💭 **Suy nghĩ: Vì sao khi khám phá một bộ dữ liệu mới, xem nhiều biểu đồ cùng lúc (dashboard) lại hữu ích hơn xem từng biểu đồ riêng lẻ?**

Dashboard cho thấy cùng lúc hình dạng phân bố, trung bình, quan hệ 2 biến và outlier. Dễ đối chiếu: ví dụ bar nói Toán TB cao, boxplot lại chỉ ra một điểm 1.0 kéo cảm nhận, scatter nói Toán–Văn không đi cùng nhau. Nếu xem từng hình cách nhau dễ quên ngữ cảnh. Khi khám phá bộ dữ liệu mới, 1 hình 4 góc nhìn giúp quyết định bước làm sạch/chuẩn hóa tiếp theo nhanh hơn.

🔧 Cải tiến: đổi lưới plt.subplots(2, 2, ...) thành lưới 1 hàng x 3 cột (plt.subplots(1, 3, figsize=(15,4))), chỉ giữ 3 biểu đồ: bar, scatter, histogram:

✏️ *Dán code dashboard 1×3 đã cải tiến của bạn vào đây*

```
import pandas as pd

import matplotlib.pyplot as plt

diem_lop = {
   "ten": ["An","Binh","Chi","Dung","Em","Phuc","Giang","Hoa"],
   "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0, 6.5, 9.5],
   "van":  [7.0, 8.0, 8.5, 6.5, 7.5, 9.0, 6.0, 8.0],
}
df = pd.DataFrame(diem_lop)

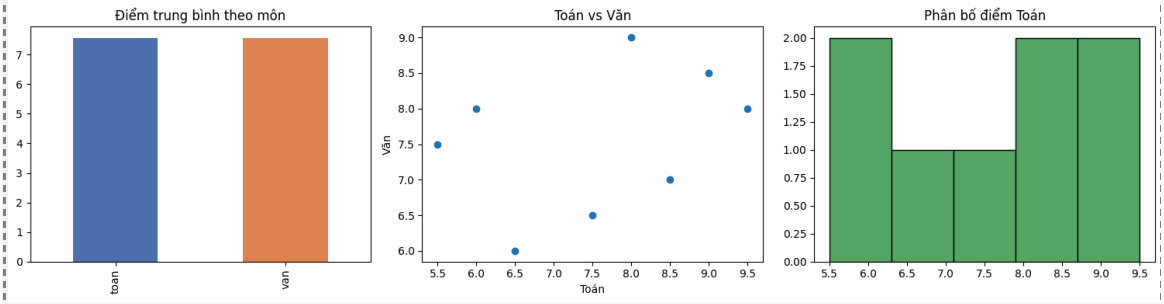
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

df[["toan","van"]].mean().plot(kind="bar", ax=axes[0], color=["#4C72B0","#DD8452"])
axes[0].set_title("Điểm trung bình theo môn")

axes[1].scatter(df["toan"], df["van"])
axes[1].set_title("Toán vs Văn")
axes[1].set_xlabel("Toán")
axes[1].set_ylabel("Văn")

axes[2].hist(df["toan"], bins=5, color="#55A868", edgecolor="black")
axes[2].set_title("Phân bố điểm Toán")

plt.tight_layout()
plt.show()
```



Mức độ tự tin với chủ đề buổi này (khoanh tròn): 1 — 2 — 3 — 4 — 5

BÁO CÁO TRẢI NGHIỆM CUỐI BUỔI

Hoàn thành mục này trước khi xuất PDF và nộp bài:

1. TRẢI NGHIỆM — Tôi đã chạy/thử nghiệm đoạn code/bài toán nào? Kết quả quan sát được là gì?

Đã chạy đủ 4 demo matplotlib (bar, scatter, hist, line), MinMaxScaler rồi đổi StandardScaler, get_dummies 1 cột rồi 2 cột, quy trình bán hàng 3 bước + thêm 2 sản phẩm + histogram, boxplot+IQR, LabelEncoder vs One-Hot vs Categorical, và dashboard 2×2 rồi 1×3. Quan sát nổi bật: (1) Kiên toán=1.0 bị IQR bắt đúng; (2) Cà phê giá 85.000 làm MinMax của các mặt hàng khác bị dồn gần 0; (3) LabelEncoder xếp 'Giỏi' = 0 vì G đứng trước trong alphabet — trái với trực giác học lực.

2. QUAN SÁT – PHẢN ÁNH — Kết quả có giống dự đoán ban đầu của tôi không? Điều gì khiến tôi bất ngờ hoặc chưa hiểu rõ?

Dự đoán ban đầu: Toán cao thì Văn cũng cao — scatter không ủng hộ. Bất ngờ thứ hai: StandardScaler ra số âm, lúc đầu tưởng 'sai' vì điểm không thể âm, rồi hiểu đó là khoảng cách tính bằng số sigma so với trung bình. Chưa thật sự tự tin phần chọn scaler nào cho từng mô hình (khi nào MinMax, khi nào Standard, khi nào RobustScaler vì outlier).

3. KHÁI QUÁT HÓA — Vì sao cần chọn đúng loại biểu đồ và vì sao cần chuẩn hóa/mã hóa dữ liệu trước khi đưa vào mô hình AI? (diễn đạt bằng lời của chính mình)

Biểu đồ sai loại dễ đọc sai câu chuyện: dùng bar để hỏi xu hướng theo tuần thì không thấy dip tuần 5 rõ như line; dùng bảng số thì không thấy outlier như boxplot. Chuẩn hóa để các cột khác đơn vị không 'át' nhau. Mã hóa để máy tính nhận biến chữ như tín hiệu số mà không bịa ra thứ tự giả. Hai bước này thuộc tiền xử lý — làm trước fit(), không phải trang trí.

4. ÁP DỤNG – RÚT KINH NGHIỆM — Nếu gặp một bộ dữ liệu mới, tôi sẽ bắt đầu khám phá nó như thế nào? Tôi còn thắc mắc điều gì?

Thứ tự sẽ làm: (1) df.head()/info()/describe() và đếm missing; (2) histogram + boxplot từng cột số, bar cho cột chữ; (3) xử lý missing và outlier có chủ đích; (4) One-Hot hoặc Categorical tùy biến có thứ tự hay không; (5) scale sau khi đã tách train/test (fit scaler trên train); (6) mới đưa vào mô hình. Thắc mắc còn lại: có nên scale biến One-Hot không, và với giá tiền lệch phải thì log-transform trước rồi mới scale có tốt hơn không.

Mức độ tự tin với chủ đề buổi này (khoanh tròn): 1 — 2 — 3 — 4 — (5)

✅ Sau khi hoàn tất, vào File → Download → PDF Document để xuất tài liệu này (kèm toàn bộ code cải tiến, ảnh biểu đồ và ghi chú của bạn) và nộp theo hướng dẫn của giảng viên.